

Assignment:

Budget Buddy Student Expense Tracker

Scenario

You are a **Junior Developer** at a fintech startup in Myanmar. Your team is developing **Budget Buddy**, a mobile application designed to help university students manage their daily spending.

Currently, the expense tracking module is having critical issues. Students are accidentally entering negative numbers as expenses, which incorrectly increases their remaining budget. Additionally, some users are logging single purchases that are larger than their entire monthly allowance, causing the system to display confusing negative balances.

As the developer, you must fix this by creating an **Expense Validation System**. You need to ensure that every recorded expense is checked against common-sense financial rules before the budget is updated.

The Code Foundation

Learners are provided with the following Python function, which acts as the "logic layer" of the application. Your goal is to verify that this logic is airtight before it is deployed to students.

budget_logic.py

```
def validate_expense(amount, budget):  
    # Rule 1: Check if the expense is a positive number  
    if amount <= 0:  
        return False  
  
    # Rule 2: Check if the user has enough budget remaining  
    if amount > budget:  
        return False  
  
    # If both rules are passed, allow the entry  
    return True
```

Assignment:

Budget Buddy Student Expense Tracker

Assignment Instructions

Task 1: Create the Test Plan

Create a test plan for three test cases, Normal Expense, Negative Entry, and Over Budget.

Test Case 1: A student makes a standard, affordable purchase.

Test Case 2: A student accidentally enters a negative value or zero.

Test Case 3: A student tries to spend more than their remaining monthly allowance.

- **Identify the Scenarios:** Before writing code, list the "Happy Path" (valid data) and "Edge Cases" (invalid or extreme data).
- **Define Test Data:** Choose specific numbers for amount and budget that specifically trigger the rules you wrote.
- **Provide Description:** Write detailed description on what you will test.
- **Determine Expected Results:** Pass or Fail.
- **Format the Table:** Use a structured format to make the plan readable for other developers or QA testers.

Task 2: Writing Unit Tests

Use the unittest library to automate the verification of the rules defined in Task 1.

- **Import Necessary Tools:** Start by importing the unittest module and the function you want to test.
- **Create a Test Class:** Define a class that inherits from unittest.TestCase. This "wraps" all your tests together.
- **Write Test Methods:** Every method must start with the word test_ (e.g., test_valid_expense).
- **Use Assertions:** Use self.assertTrue() or self.assertFalse() to check if the function's actual result matches your expectation.

Assignment:

Budget Buddy Student Expense Tracker

- **Execute the Suite:** Add the if `__name__ == "__main__":` block to allow the script to run the tests directly from the terminal.

Task 3: Write explanations for the outputs of each test case

- **Analyze the Results:** Explain what the execution symbols mean, where a **dot (.)** indicates a successful pass and an **F** signals a logic failure.
- **Confirm the Summary:** Interpret the total count of tests run and the final status (**OK** or **FAILED**) to determine if the code is ready for the app.
- **Map to Business Logic:** For each test, describe how the output successfully enforced a financial rule, such as blocking a negative number or preventing overspending.

What to Submit

To complete this task, you must submit the following:

- **budget_logic.py:** Your core Python file containing the `validate_expense` function.
- **test_budget.py:** Your testing script containing the `unittest` class and three test methods.
- **Test Report:** A brief explanation of your 3 test cases and a screenshot of your terminal showing the "OK" status after running the tests.

Do Not Submit

- **Unfinished Code:** Do not submit the logic file if it still contains syntax errors.
- **Virtual Environments:** Do not include `.env` or `__pycache__` folders in your submission.
- **Personal Data:** Do not include actual personal financial data in your test cases; use the provided mock data from the test plan.

Assignment:

Budget Buddy Student Expense Tracker

Marking Rubric (100 Marks)

Criteria	Excellent (Full Marks)	Satisfactory (Partial)	Needs Improvement (Low)	Score
Task 1: Test Plan	30 Marks: All 3 cases (Normal, Negative, Over) are clearly defined with accurate data, descriptions, and expected results. Table is professional.	20 Marks: Test cases are present but data is vague or the distinction between "Pass" and "Fail" result is inconsistent.	10 Marks: Table is missing or incomplete. Scenarios do not match the required test cases.	30
Task 2: Unit Testing	30 Marks: Script correctly imports unittest, uses a class, includes 3 methods starting with test_, and uses correct assertions (assertTrue/False).	20 Marks: Code runs but has minor naming issues (e.g., missed test_ prefix) or uses inappropriate assertion types.	10 Marks: Code contains syntax errors, fails to run, or does not use the unittest library.	30
Task 3: Output Explanation	20 Marks: Clear 3-line explanation identifying symbols (., F), confirming the OK	10 Marks: Explanation is present but fails to explain the symbols or the business	0 Marks: No explanation provided or explanation is unrelated to the	20

Assignment:

Budget Buddy Student Expense Tracker

Criteria	Excellent (Full Marks)	Satisfactory (Partial)	Needs Improvement (Low)	Score
	status, and mapping results to fintech rules.	impact of the test results.	actual code output.	
Code Quality (Task 4/5 Concepts)	10 Marks: Code includes a clear docstring and logical comments (#) explaining the validation rules.	5 Marks: Minimal documentation; comments are sparse or only restate the code.	0 Marks: No comments or docstrings present in the submission.	10
Submission Requirements	10 Marks: All files (budget_logic.py, test_budget.py, Report) submitted correctly. No junk files (__pycache__).	5 Marks: Missing one file or included unnecessary folders like __pycache__.	0 Marks: Major files missing or code does not execute.	10
TOTAL				100